ineffective for us. We found that it was easy to exploit a PRM trained on this dataset via even naïve strategies such as best-of-N sampling. We hypothesize that this is likely a result of the distribution shift between the GPT-4 generated samples in their dataset and our PaLM 2 models. Rather than proceeding with the expensive process of collecting crowd-worker PRM labels for our PaLM 2 models, we instead apply the approach of Wang et al. [45] to supervise PRMs without human labels, using estimates of per-step correctness obtained from running Monte Carlo rollouts from each step in the solution. Our PRM's per-step predictions therefore correspond to value estimates of reward-to-go for the base model's sampling policy, similar to recent work [31, 45]. We also compared to an ORM baseline (Appendix F) but found that our PRM consistently outperforms the ORM. Hence, all of the search experiments in this section use a PRM model. Additional details on PRM training are shown in Appendix D.

Answer aggregation. At test time, process-based verifiers can be used to score each individual step in a set of solutions sampled from the base model. In order to select the best-of-N answers with the PRM, we need a function that can aggregate across all the per-step scores for each answer to determine the best candidate for the correct answer. To do this, we first aggregate each individual answer's per-step scores to obtain a final score for the full answer (step-wise aggregation). We then aggregate across answers to determine the best answer (inter-answer aggregation). Concretely, we handle step-wise and inter-answer aggregation as follows:

- **Step-wise aggregation.** Rather than aggregating the per-step scores by taking the product or minimum [22, 45], we instead use the PRM's prediction at the last step as the full-answer score. We found this to perform the best out of all aggregation methods we studied (see Appendix E).
- **Inter-answer aggregation.** We follow Li et al. [21] and apply "best-of-N weighted" selection rather than standard best-of-N. Best-of-N weighted selection marginalizes the verifier's correctness scores across all solutions with the same final answer, selecting final answer with the greatest total sum.

5.2. Search Methods Against a PRM

We optimize the PRM at test time via search methods. We study three search approaches that sample outputs from a few-shot prompted base LLM (see Appendix G). An illustration is shown in Figure 2.

Best-of-N weighted. We sample N answers independently from the base LLM and then select the best answer according to the PRM's final answer judgement.

Beam search. Beam search optimizes the PRM by searching over its per-step predictions. Our implementation is similar to BFS-V [10, 48]. Concretely, we consider a fixed number of beams N and a beam width M. We then run the following steps:

1. sample N initial predictions for the first step in the solution
2. score the generated steps according to the PRM's predicted step-wise reward-to-go estimate (which also corresponds to the total reward from the prefix since the reward is sparse in this setting)
3. filter for only the top $\frac{N}{M}$ highest scoring steps
4. now from each candidate, sample M proposals from the next step, resulting in a total of $N/M \times M$ candidate prefixes again. Then repeat steps 2-4 again.

We run this algorithm until the end of a solution or the maximum number of rounds of beam expansion are attained (40 in our case). We conclude the search with N final answer candidates, to which we apply best-of-N weighted selection described above to make our final answer prediction.

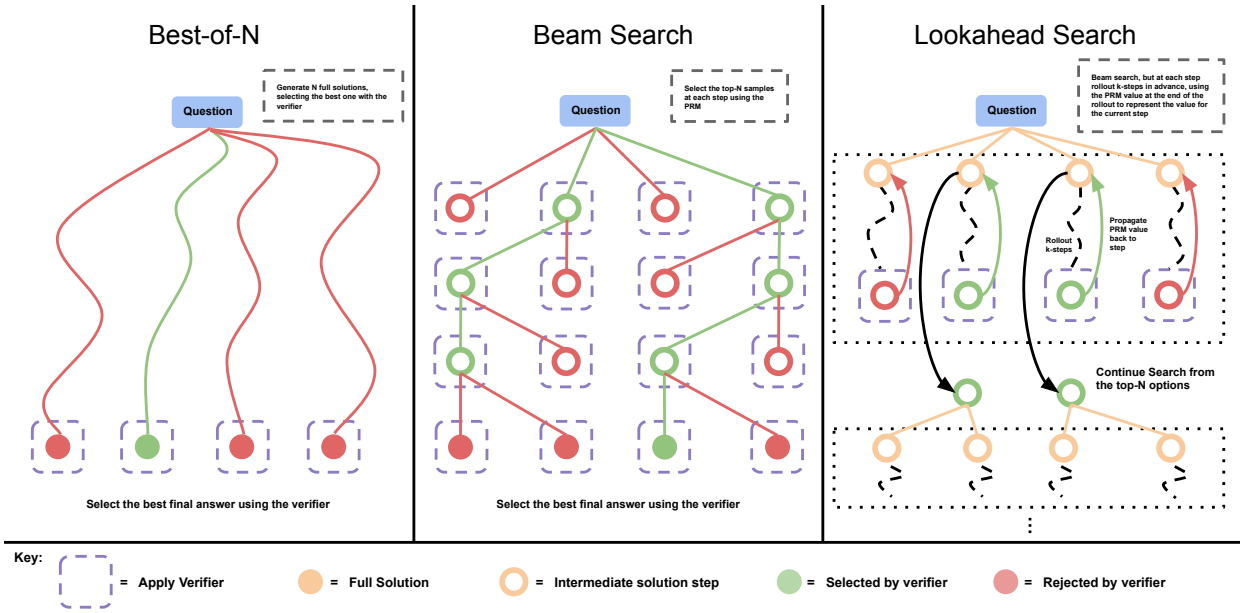

Figure 2 | ***Comparing different PRM search methods.*** **Left:** Best-of-N samples N full answers and then selects the best answer according to the PRM final score. **Center:** Beam search samples N candidates at each step, and selects the top M according to the PRM to continue the search from. **Right:** lookahead-search extends each step in beam-search to utilize a k-step lookahead while assessing which steps to retain and continue the search from. Thus lookahead-search needs more compute.

Lookahead search. Lookahead search modifies how beam search evaluates individual steps. It uses lookahead rollouts to improve the accuracy of the PRM's value estimation in each step of the search process. Specifically, at each step in the beam search, rather than using the PRM score at the current step to select the top candidates, lookahead search performs a simulation, rolling out up to k steps further while stopping early if the end of solution is reached. To minimize variance in the simulation rollout, we perform rollouts using temperature 0. The PRM's prediction at the end of this rollout is then used to score the current step in the beam search. That is, in other words, we can view beam search as a special case of lookahead search with $k = 0$. Given an accurate PRM, increasing k should improve the accuracy of the per-step value estimates at the cost of additional compute. Also note that this version of lookahead search is a special case of MCTS [38], wherein the stochastic elements of MCTS, designed to facilitate exploration, are removed since the PRM is already trained and is frozen. These stochastic elements are largely useful for learning the value function (which we've already learned with our PRM), but less useful at test-time when we want to exploit rather than explore. Therefore, lookahead search is largely representative of how MCTS-style methods would be applied at test-time.

5.3. Analysis Results: Test-Time Scaling for Search with Verifiers

We now present our results comparing various search algorithms and identify a prompt difficulty dependent compute-optimal scaling strategy for search methods.

Comparing search algorithms. We first conduct a sweep over various search settings. In addition to the standard best-of-N approach, we sweep over the two main parameters that distinguish different tree-search methods: beam-width M and number of lookahead steps k. While we are not able to extensively sweep every single configuration, we sweep over the following settings with a maximum budget of 256:

1) Beam search with the beam width set to $\sqrt{N}$, where N is the generation budget.
2) Beam search with a fixed beam width of 4.

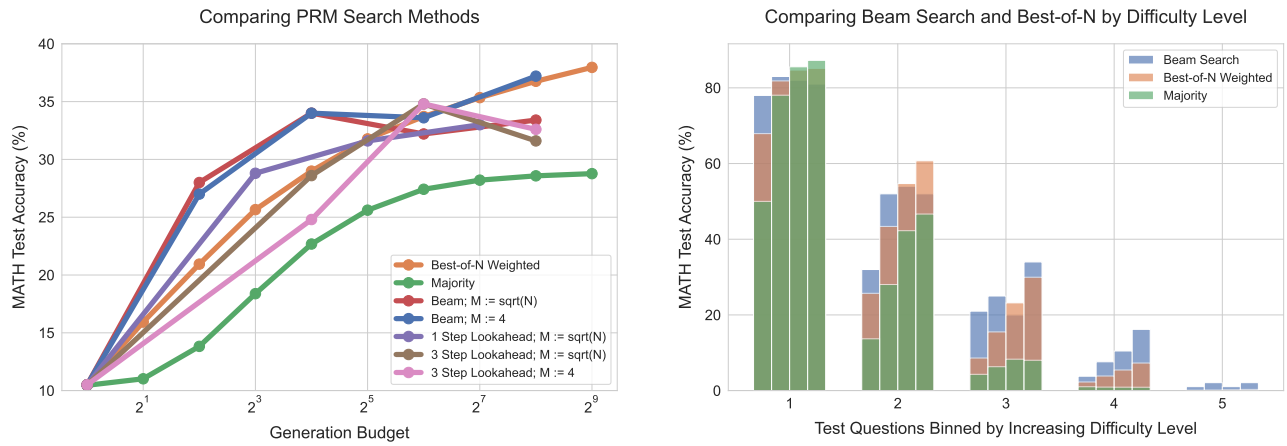

Figure 3 | **Left:** ***Comparing different methods for conducting search against PRM verifiers***. We see that at low generation budgets, beam search performs best, but as we scale the budget further the improvements diminish, falling below the best-of-N baseline. Lookahead-search generally underperforms other methods at the same generation budget. **Right:** ***Comparing beam search and best-of-N binned by difficulty level***. The four bars in each difficulty bin correspond to increasing test-time compute budgets (4, 16, 64, and 256 generations). On the easier problems (bins 1 and 2), beam search shows signs of over-optimization with higher budgets, whereas best-of-N does not. On the medium difficulty problems (bins 3 and 4), we see beam search demonstrating consistent improvements over best-of-N.

3) Lookahead search with $k = 3$ applied to both beam-search settings 1) and 2).
4) Lookahead search with $k = 1$ applied to beam-search setting 1).

To compare search methods as a function of generation budget fairly, we build a protocol for estimating the cost of each method. We consider a generation to be a sampled answer from the base LLM. For beam search and best-of-N the generation budget corresponds to the number of beams and N respectively. Lookahead search, however, utilizes additional compute: at each step of the search, we sample k additional steps ahead. Therefore, we define the cost of lookahead-search to be $N \times (k + 1)$ samples.

Results. As shown in Figure 3 (left), with smaller generation budgets, beam search significantly outperforms best-of-N. However, as the budget is scaled up, these improvements greatly diminish, with beam search often underperforming the best-of-N baseline. We also see that, lookahead-search generally underperforms other methods at the same generation budget, likely due to the additional computation inducted by simulating the lookahead rollouts. The diminishing returns from search are likely due to exploitation of the PRM's predictions. For example, we see some instances (such as in Figure 29), where search causes the model to generate low-information repetitive steps at the end of a solution. In other cases, we find that over-optimizing search can result in overly short solutions consisting of just 1-2 steps. This explains why the most powerful search method (i.e., lookahead search) underperforms the most. We include several of these examples found by search in Appendix M.

Which problems does search improve? To understand how to compute-optimally scale search methods, we now conduct a difficulty bin analysis. Specifically, we compare beam-search ($M = 4$) against best-of-N. In Figure 3 (right) we see that while in aggregate, beam search and best-of-N perform similarly with a high generation budget, evaluating their efficacy over difficulty bins reveals very different trends. On the easy questions (levels 1 and 2), the stronger optimizer of the two approaches, beam search, degrades performance as the generation budget increases, suggesting signs of exploitation of the PRM signal. In contrast, on the harder questions (levels 3 and 4), beam search consistently outperforms best-of-N. On the most difficult questions (level 5), no method makes much meaningful progress.